

Detecting Python Type Errors through Interprocedural Type-State Analysis

ANONYMOUS AUTHOR(S)

Python type errors depend on the state in which an operation executes. A function call can change the type of an object attribute or a global variable, making a previously valid operation incompatible with the updated state. Detecting such errors requires connecting input conditions to the state changes established across calls. We present an interprocedural type-state analysis for Python type-error detection. Its central model relates function entries to ordered operation requirements, state updates, and exits, preserving both conditional effects and intermediate failures. The analysis constructs these relations by sharing facts independent of unknown entry types and using a large language model to propose implicit type alternatives. At each call, it instantiates object identities and propagates the corresponding state changes. A conflict triggers selective expansion of the relevant callee type paths and contextual re-inference; revised relations then update dependent callers. The analysis separately reports runtime type errors and annotation-contract errors. We evaluate paired detection on xxx recent GitHub bug-fix records, diagnostic precision, component effects, and total analysis cost. The evaluation reports xxx paired runtime detections and xxx paired annotation-contract detections, with respective diagnostic precision of xxx% and xxx%.

Additional Key Words and Phrases: Python, type errors, type-state relations, interprocedural analysis, function summaries

1 Introduction

Python’s dynamic type system offers developers considerable flexibility and enables rapid development. However, it also allows type errors to remain undetected until runtime. These errors can interrupt program execution and cause application failures, compromising software reliability. Consequently, substantial research effort has been devoted to static type-error detection for Python, with the aim of identifying such errors before execution.

Existing approaches have made progress in static type-error detection for Python. Type checkers such as Mypy and Pyrely use type annotations to check assignments, function calls, and other operations against declared types, and have been adopted in major production codebases and open-source projects, including Dropbox, Instagram, PyTorch, and JAX [2, 7]. Automatic type inference further improves detection. For example, Pyinder detected 50% (34/68) of real-world type errors in its benchmark [13]. However, these approaches rely on static type models that do not fully capture how types vary with execution conditions or change through function calls. As a result, they can produce false alarms or miss type errors. In Pyinder’s evaluation, such limitations accounted for 31% of the confirmed false alarms in its inspected sample [13]. This gap between static type models and Python’s dynamic typing behavior limits the effectiveness of existing type-error detection methods.

To address this gap, we aim to improve Python type-error detection by tracking type changes across function calls. A function’s type requirements concern not only its arguments but also the global variables, instance attributes, and other values it accesses. We refer to their collective type information at a given program point as the *type-state*. Our approach captures how a function transforms this state under different execution conditions and uses these transformations to check subsequent operations. To achieve this goal, we need to address three key challenges.

Challenge 1: Modeling type-state changes across function calls. A function can change the caller’s type-state not only through its return value but also by updating global variables or attributes of mutable objects. These updates depend on the function’s execution path, so calls to the same function can establish different type-states. Conventional type annotations describe parameter,

return, and attribute types, but do not specify how a call transforms the incoming type-state under different execution conditions. We therefore need a new model to express these conditional type-state changes and their effects on subsequent operations.

Challenge 2: Extracting conditional type-state changes from code. Program execution involves both explicit and implicit paths. Explicit paths arise from branch statements, while implicit paths arise from Python operations whose behavior depends on the types involved. For example, addition can perform arithmetic, concatenate strings, or invoke a user-defined method, with different type requirements and effects in each case. Analyzing these paths across function calls further requires considering combinations of caller and callee paths, leading to path explosion. Accounting for implicit paths and managing path explosion therefore make it challenging to extract conditional type-state changes from code.

Challenge 3: Detecting type errors using conditional type-state changes. Due to limitations in analysis precision, the extracted type-state transition relations may contain inaccuracies, which can cause valid operations to be flagged as type errors. A detected type conflict therefore does not necessarily indicate an error in the program. Distinguishing a real error from an analysis inaccuracy requires examining how the conflicting state arises under the relevant calling conditions. Moreover, an inaccurate type-state change can propagate through multiple calls, affecting error detection in other functions. The challenge is therefore to distinguish type errors from false alarms and correct analysis inaccuracies.

To address these challenges, we propose PyTT(Python Type Tracker), a Python type-error detector that models and tracks conditional type-state transitions across function calls. To address **Challenge 1**, we introduce a *path-sensitive type contract* (PSTC) to represent these transitions, capturing a function’s type requirements and updates along different execution paths. To address **Challenge 2**, PyTT uses static analysis to identify explicit paths and a large language model (LLM) to infer implicit paths. It further uses static analysis to simplify code whose type behavior is independent of the calling context, reducing repeated analysis across paths to mitigate path explosion. To address **Challenge 3**, PyTT uses PSTCs to trace detected conflicts across function boundaries. It analyzes the resulting traces to distinguish type errors from false alarms. For false alarms, PyTT corrects the inaccuracies in the corresponding contracts and updates the affected analysis results.

We evaluate PyTT on xxx real-world type-error fixes from xxx Python repositories, measuring detection effectiveness, diagnostic precision, and analysis cost. PyTT detects xxx runtime type errors and xxx annotation-contract errors, with precision of xxx% and xxx%, respectively. Each detection is checked against both the buggy and fixed revisions. Ablation experiments examine the contributions of conditional type-state modeling, explicit and implicit path analysis, and conflict analysis and correction. We further evaluate PyTT on xxx independent repositories, where it identifies xxx previously unknown type errors.

The contributions are as follows:

- We introduce the path-sensitive type contract (PSTC), a model that captures conditional type-state transitions across function calls for Python type-error detection.
- We propose PyTT, a type-error detection method that combines static analysis and LLM-based analysis to extract these transitions. PyTT analyzes conflict traces across functions to distinguish type errors from false alarms and correct analysis inaccuracies.
- We evaluate PyTT on real-world type-error fixes and independent Python repositories, assessing detection effectiveness, diagnostic precision, and analysis cost, and examining the contributions of its key components through ablation experiments.

2 Background and Motivating Example

In this section, we first introduce the key concepts underlying our approach, including the Python language features and the Hoare-Triple. We then illustrate why type-error detection must track type-state changes across these calls through a motivating example.

2.1 Dynamic Typing and Duck Typing in Python

Type-error detection first determines the types of the operands at an operation and then checks whether the operation supports those types [13]. In Python, however, both steps are complicated by its flexible type system. Because Python is dynamically typed, the types that reach an operation can change during execution. Even when these types are known, duck typing requires compatibility to be judged by the behavior provided by the operands rather than by their class names alone. Moreover, different operand types may invoke different implementations of an operation and produce results of different types, which further affects the types that reach subsequent operations.

More specifically, the **dynamic typing** feature of Python allows a variable, `XXXXXXXXXX`, to be bound to objects of different types during program execution [15, 16]. Consequently, the type of an operand at an operation depends on the assignments executed before that operation. These changes are not xxx within a single function: a call may return a new value, update a global variable, or modify an attribute of an object shared with its caller. Therefore, determining the types at an operation requires tracking changes both within a function and across function calls.

Duck typing further affects both compatibility checking and type tracking. In Python, an object is compatible with an operation if it provides the required behavior, regardless of its nominal class [17]. Thus, objects of different classes may support the same operation, while the same operator may invoke different implementations for different operand types. A detector must identify the methods or protocols supported by the operands to determine whether the operation is valid. At the same time, the selected implementation may produce a different return type or state change, which must be tracked when checking subsequent operations. Type annotations can describe the possible operand types or their required interfaces [21]. However, type-error detection must further identify the correspondence between the input and output types of an operation. This correspondence indicates whether the operation is valid for a given combination of operand types and, if so, what type of result it produces.

2.2 Hoare Triples as Function Summaries

Because a function call may change the types subsequently observed by its caller, detecting type errors across calls requires propagating these changes from the callee to the caller. A function summary can provide this information without reproducing the callee's implementation at each call site. To support type-error detection, the summary must describe not only the conditions before the call but also the type changes established after it. Hoare triples provide a standard way to express this before-and-after relationship.

Hoare logic describes how executing a program changes its state [5, 14]. A Hoare triple has the form

$$\{P\} C \{Q\}, \quad (1)$$

where P is a precondition, C is a command, and Q is a postcondition. The triple states that if P holds before C executes and C terminates normally, then Q holds afterward. For example, P may state that `msg.body` has type `str`; C may encode and update the body; and Q may state that `msg.body` has type `bytes`. The triple therefore describes both the state required before a command and the state established after it.

A Hoare triple can therefore summarize a function without copying its body to every call site. While its precondition records what must hold when the function is called, its postcondition records the return value and any changes visible to the caller. When a call occurs, the function's parameters are matched with the caller's arguments, after which the postcondition can be used to check the operations following the call. Moreover, Hoare triples can be composed: if $\{P\} C_1 \{R\}$ and $\{R\} C_2 \{Q\}$ hold, the state R produced by C_1 becomes the state used to analyze C_2 .

This form of summary is particularly useful for Python type-error detection. Although a function signature describes parameter and return types, it may not show that the function changes a global variable or an attribute of an argument. A Hoare triple can capture such changes in its postcondition and then apply them at the call site. As a result, the caller obtains the updated type state needed to check its next operation. Hoare triples are therefore suitable for summarizing and propagating type changes across Python function calls.

2.3 A Call That Changes an Operation's Validity

Figure 1 illustrates why type-error detection requires tracking the state established by a call. The Message class declares `body: str | bytes`. Both encoders accept a string. The `encode` method of the text encoder returns that string; the UTF-8 encoder returns bytes. The helper `encode_message` writes the encoder's result back to the same message object and implicitly returns None.

Function with Type Error

```
def make_line(msg: Message):
    if not isinstance(msg.body, str):
        raise ValueError("...")
    1 #msg.body: str
    encode_message(msg, Utf8Encoder())
    3 #msg.body: bytes
    print(msg.body + '\n')
    #TypeError: can't concat str to bytes
```

Function with Complex Type Transformation

```
def encode_message(msg: Message,
                  encoder: TextEncoder|Utf8Encoder):
    if isinstance(msg.body, str):
        msg.body = encoder.encode(msg.body)
    2 #encoder: TextEncoder => msg.body: str
    #encoder: Utf8Encoder => msg.body: bytes
```

Related Classes

```
class Message:
    body: str | bytes

class TextEncoder:
    def encode(self, text: str) -> str:
        return text

class Utf8Encoder:
    def encode(self, text: str) -> bytes:
        return text.encode("utf-8")
```

Detection Results

Mypy, Pyrefly, Pyright: **No diagnostic**

PyTT: **Type error detected**

Fig. 1. A call changes the type state used by a subsequent operation. Point 1 establishes a string body; point 2 relates the selected encoder to the field update; point 3 has a bytes body, making concatenation with a string fail. The lower-right panel reports the illustrated example's outcomes for Mypy, Pyrefly, Pyright, and our approach, PyTT.

From the guard to the update. In `make_line`, a non-string body triggers `ValueError`. Every execution reaching point 1 has therefore passed the guard with a string body. The subsequent call supplies that message and a `Utf8Encoder` instance to the helper. At point 2, the helper's two encoder cases connect the input choice to different updates: the text encoder establishes a string body and the UTF-8 encoder establishes bytes. Both writes satisfy the field's union annotation. The relevant difference is the state that each call establishes for subsequent operations.

From the update to the failure. Instantiating the UTF-8 case at the shown call binds the helper's message to the caller's object. Consequently, point 3 contains a bytes body. The string fact at point 1

concerns the old contents and does not constrain the updated field. Evaluation of `msg.body + '\n'` then raises `TypeError` before `print` is invoked. With `make_line` as the analysis entry, that exception escapes the function. Using `TextEncoder` at the same call would instead establish a string body and permit the concatenation.

This reasoning distinguishes three representations. The helper’s return type, `None`, gives no description of the field update. A set of possible field types, `str | bytes`, captures the possible contents. A relation between the selected encoder, incoming body, and outgoing body identifies the contents established in the actual calling context. Preserving that relation lets the analyzer invalidate the old field fact and check the addition using its new value.

The result panel in Figure 1 identifies our approach as *PyTT*. For the illustrated example, it records no diagnostic from *Mypy*, *Pyre*fly, or *Pyright* and a type error detected by *PyTT*; the associated tool versions and configurations are xxx. The mechanism illustrated by this result is the propagation of the encoder-dependent write to the operation that consumes the field. Section 4 evaluates detection across the benchmark under the stated configurations.

3 Approach

This section introduces our solution for precise interprocedural Python type-error detection by modeling and tracking conditional type-state changes.

3.1 Overview

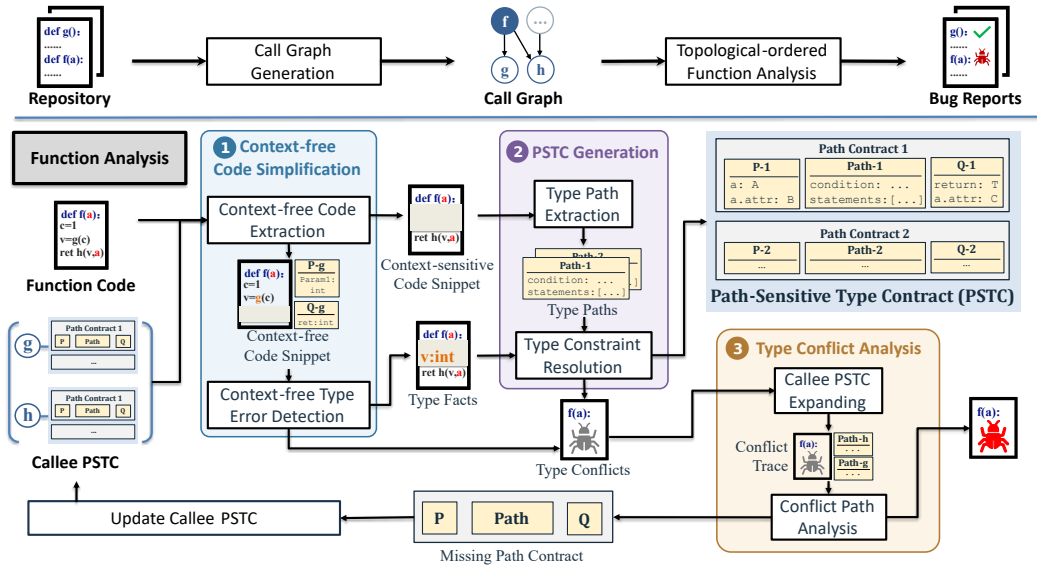


Fig. 2. Overview of PyTT’s workflow

Figure 2 illustrates the workflow of PyTT for interprocedural Python type-conflict detection. Given a Python repository, PyTT constructs a call graph and analyzes functions in callee-first order. For each function, PyTT identifies type errors and constructs a **path-sensitive type contract (PSTC)** to represent the type-state transitions within the function, based on its source code and the PSTCs of its callees. The constructed PSTC is subsequently used to analyze callers and refined as detected conflicts are investigated. This per-function procedure consists of three major phases.

PyTT first separates and analyzes computations independent of the function’s entry type-state, then summarizes the remaining context-sensitive behavior as a PSTC, and finally investigates the resulting type conflicts to either confirm type errors or refine incomplete PSTCs.

(1) Context-free Code Simplification. This phase is designed to avoid repeatedly analyzing computations whose type behavior can be determined independently of the function’s entry type-state. PyTT separates these computations from the context-sensitive remainder, checks them once, and propagates the resulting type facts to their dependent operations. In Figure ??, the previously constructed PSTC for g allows PyTT to infer $v: \text{int}$ from $c = 1$ and $v = g(c)$. The remaining call $h(v, a)$ can therefore be analyzed with the known type of v , while leaving the entry-dependent type of a to the next phase. The original checks and effects of the separated computations remain part of the function’s summarized behavior.

(2) PSTC Generation. This phase summarizes how the context-sensitive part of the function behaves under different entry type-states. PyTT extracts explicit execution paths and resolves their type constraints using the type facts produced in the first phase and the available PSTCs of callees. For behavior not fully exposed by the extracted paths, an LLM helps identify implicit type alternatives and their corresponding entry and exit assertions. PyTT combines these assertions with the ordered checks and effects of each path to construct the function’s PSTC. In the running example, this phase analyzes $h(v, a)$ using $v: \text{int}$ together with the possible entry types of a . The resulting path contracts are propagated to callers, whereas conflicts encountered during their construction or composition are passed to the third phase.

(3) Type Conflict Analysis. A detected conflict may indicate either an incompatible program behavior or an incomplete PSTC. To distinguish between the two, PyTT expands the relevant paths from the callee PSTCs, connects them to the caller path leading to the conflicting call site, and re-analyzes the resulting conflict trace under the calling context. If the incompatibility remains, PyTT reports a type error. Otherwise, it derives a missing or revised path contract and uses it to refine the corresponding callee PSTC. For example, a conflict at $h(v, a)$ is analyzed together with the relevant behavior of h and the calling facts established for v and a , rather than being reported directly.

The three phases form an iterative interprocedural analysis. Whenever Type Conflict Analysis refines a callee PSTC, PyTT invalidates the dependent results and re-enqueues the affected callers. This process continues until the relevant PSTCs stabilize or the configured exploration budget is reached. PyTT then reports the confirmed type errors and records any remaining undecided cases as unresolved.

3.2 Design of Path-Sensitive Type Contracts

As discussed earlier, the behavior of a Python function may depend on the types of not only its parameters but also the surrounding program state, such as global variables and object attributes. Conversely, executing the function may update these entities and thereby change their types. Because such updates may differ across control-flow paths and calling contexts, a single input–output type signature cannot precisely capture these type-state dependencies. To represent them, PyTT models each function using a Hoare-triple-like abstraction called a *Path-Sensitive Type Contract* (PSTC).

We define the *type-state* at a program point as a mapping $S : V \rightarrow T$, where V and T denote the universes of variables and types, respectively. The variables in V include function parameters, global variables, class attributes, object attributes, and the parameters and return value.

For a function f , let P denote the set of admissible entry type-states, M the set of feasible execution paths through f and Q the set of reachable exit type-states. We define the path-sensitive type

contract of f as a relation over these three sets:

$$PSTC(f) \subseteq P \times M \times Q.$$

A triple $(p, m, q) \in PSTC(f)$ states that an execution of f starting from entry type-state p will follow path m and terminate in exit type-state q . Thus, the relation records not only the possible entry and exit states, but also which execution paths connect them.

At a call site, the pair (p, q) summarizes the type-state effects of the invocation along path m , while m records the control-flow behavior responsible for these effects. PyTT uses (p, q) to propagate type-state transition across functions. If this propagation produces a type conflict, PyTT uses the associated path m as behavioral evidence for subsequent analysis. By representing distinct feasible executions as separate triples, a PSTC preserves the correspondence among the required entry state, the executed path, and the resulting exit state.

3.3 Phase I: Context-free Code Simplification

Context-free Code Extraction. The figure’s term *context-free* means independent of the unknown entry types and state. A computation belongs to this part, C_I , when local semantics and available callee PSTCs determine its type behavior without instantiating that unknown information. The remaining computations form the context-dependent part C_D , labeled *Context-sensitive Code Snippet* in the figure. Dependence follows values, object state, guards, and callee effects. Both parts retain source positions and control dependencies for assembly in execution order.

For the schematic function $f(a)$, the literal assignment $c = 1$ establishes an integer input to g . The depicted callee contract has an integer entry and integer return, allowing the analyzer to determine the return type of $g(c)$. The subsequent call $h(v, a)$ still depends on the unknown input a . The partition therefore separates the computation establishing $v: \text{int}$ from the use that requires additional type alternatives. Its classification uses the callee’s state dependencies and effects as well as its parameter and return types.

Context-free Type Error Detection. The analyzer checks C_I using local operation semantics and applicable callee contracts. A successful operation supplies facts to its uses in C_D ; a conflict enters Phase III. Each supplied fact records its value or location version, condition, type relation, and supporting evidence. In the figure, $v: \text{int}$ is supplied at the use of v in $h(v, a)$, allowing the computation of $g(c)$ to be shared across the candidate behaviors of a .

Entry-independent computations can contain branches, visible writes, and exceptions. Their conditions, checks, effects, and exceptional successors remain in the assembled relation. A result fact is supplied after its producing operation succeeds, and facts affected by an intervening write are updated at their later uses. Thus, simplification shares inference while retaining the execution behavior that determines each fact’s validity.

3.4 Phase II: PSTC Generation

Type Path Extraction. The analyzer explores explicit source paths in C_D within the configured bounds. Each template retains guards and operations in execution order, together with applicable facts from Phase I. The *Type Paths* shown in Figure 2 are these templates. A source path can support several implicit type behaviors: $x + y$, for example, can describe integer addition, string concatenation, or an operation implemented by repository classes. Resolving these alternatives associates each operand combination with its result and state effects.

Type Constraint Resolution. The LLM receives the explicit path, relevant use-site facts, repository type information, and available callee PSTCs. It proposes implicit input/output combinations and corresponding entry and exit assertions. The analyzer assembles them with the ordered checks and

effects from both C_I and C_D . Source paths determine the locations and ordering of the operations to which each candidate relation applies. Candidate records retain their inference context and source dependencies for subsequent inspection. The resulting cases constitute the function's PSTC.

Inferred relations participate in propagation before conflict validation. At an input state not covered by the available cases, the analyzer requests supplementary analysis or retains an unresolved outcome. An operation or annotation conflict enters Phase III, which expands and re-infers the relevant behavior in context. Uncovered inputs and conflicts therefore focus additional construction on the relations needed by current callers.

3.4.1 Call-site Composition and Operation Checks. PSTC generation and application use the same call-site composition procedure. At each call, the analyzer evaluates the call target and arguments in source order and checks argument binding. It constructs a substitution θ from formal parameters, object references, and environment locations to the calling state. Internal symbols and newly allocated objects are fresh for each instantiation. When two arguments refer to the same actual object, the corresponding formal references retain that alias.

For a case (p_i, m_i, q_i) , applicability is evaluated under

$$\text{Facts}(A) \wedge \theta(p_i). \quad (2)$$

The analyzer then processes $\theta(m_i)$ in event order. A guard refines the current condition. A read obtains the current version of its location. A write updates that location. An operation requirement is checked against the state established by the preceding events. The exit assertion projects the resulting return and shared state; it does not perform the updates a second time.

A definite write to one location representing a single object creates a strong update. For several possible targets, a weak update retains the possible old and new contents. A previous location fact survives a call when that location is provably disjoint from the possible write footprint. An unknown write invalidates precise facts for the affected region. For example, after `a.prefix = b"x"` and `b.prefix = "y"`, reading `a.prefix` yields a string when `a` is `b`, but bytes when they are distinct ordinary objects. The actual alias binding thus determines the state of a subsequent use.

For a requirement R and the current facts F , the check examines the success condition $F \wedge R$ and the failure condition $F \wedge \neg R$. A supported failure alternative becomes a candidate conflict, associated with its source location and state dependencies. Indeterminate conditions remain unresolved. Subsequent events on a normal continuation use the success facts; exceptional continuations carry the state accumulated up to the failed event. Loop and recursive expansion follows the configured budgets and records the remaining frontier.

For Figure 1, θ binds the helper's message object to the caller's message and its encoder to `Utf8Encoder`. The applicable case writes a bytes value to the body's new version. The final addition is checked using that version, producing the bytes/string conflict. The text-encoder case instead establishes a string version and satisfies the same operation requirement.

3.5 Phase III: Type Conflict Analysis

Callee PSTC Expanding. A candidate conflict links an operation or annotation check to the relations supporting its judgment. Validation begins with those dependencies and reconstructs the relevant behavior in the current calling context. The expansion unit is a *type path*: the ordered source behavior associated with a relation case's entry and exit assertions. Those assertions guide selection of the necessary callee paths on the fixed call graph.

The analyzer follows the candidate’s control, value, and alias dependencies to select callee cases. It connects their type paths to the relevant caller fragments, preserving conditions, operation requirements, writes, and exceptional control flow. The resulting *error trace* supplies both the source behavior behind the disputed relation and the call-site-specific symbolic bindings of the calling context. This error trace corresponds to the figure’s *Conflict Trace*.

Conflict Path Analysis. The analyzer re-infers behavior over the expanded trace to determine whether the conflict persists and whether the propagated relation requires revision.

The figure’s *Missing Path Contract* is a relation supplied by this analysis to supplement or correct the callee PSTC. Refinement can supplement an entry case, separate alias alternatives, restore a condition–effect association, or revise an inferred type combination. A correction derived for a particular calling condition keeps that condition. Revised summaries receive new dependency versions; results that use an older version are invalidated and the affected caller entries are re-enqueued. Analysis stops refining a candidate when its disposition is established, no new evidence is obtained, or the budget is exhausted. Algorithm 1 summarizes this propagation and feedback.

Runtime type-error reports. A runtime report requires evidence that the expanded trace reaches an operation with incompatible state and that its target protocol exception escapes the declared entry. For a repository entry, the trace includes the initialization and calls establishing the state; an open-entry report records its symbolic arguments and state assumptions. A handler that consumes the failure produces a handled-failure outcome. A corrected relation that removes the conflict produces a dismissed candidate. Insufficient evidence for either disposition produces an unresolved case. Reports retain the expanded trace, supplementary context, re-inference result, and exception disposition.

Annotation-contract reports. Existing annotations are checked against the requirements and effects inferred from function behavior. For an input annotation, the analysis examines whether the declared inputs are semantically compatible with the inferred requirements. For return and state declarations, it examines whether the inferred effects are compatible with the declared types. A conflict enters the same dependency tracing and contextual re-inference process. A confirmed report retains the declaration location, incompatible relation, and supporting behavioral evidence. Its confirmation concerns the declaration–behavior conflict and does not require a repository call that raises a runtime exception.

3.6 Reuse and Analysis Cost

Summary templates reuse extracted relations while freshening their internal symbols at each application. Reuse of an already instantiated result additionally matches the relevant entry state, aliases, call targets, semantic configuration, assumptions, remaining expansion bounds, and dependency versions. Cache keys include the transitive dependencies of reads, checks, and effects. Instances with unknown read/write footprints are recomputed. A cached failure is reconsidered under the current entry’s reachability and exception context.

The computational work comprises shared fact inference, path and type alternative construction, composition, conflict expansion, contextual re-inference, and dependency maintenance. Reuse acts at several of these points: independent facts serve multiple alternatives, event prefixes and callee templates serve multiple cases, and valid instances serve matching calls. Refinement revisits affected callers through explicit dependencies. Section 4.3 measures the complete cost and compares these forms of reuse with direct function-body analysis.

Algorithm 1 Propagation with conflict-driven relation refinement

Input: Repository P , fixed call graph G , entries E , budgets B
Output: Runtime reports, annotation-contract reports, and unresolved cases

- 1: Initialize summaries and a callee-first worklist W from G, E
- 2: **while** $W \neq \emptyset$ and analysis budget remains **do**
- 3: Remove a function entry (f, A) from W
- 4: Construct or supplement $\mathcal{S}_f$ using shared facts and type paths
- 5: Compose relations under A and collect operation and annotation conflicts
- 6: **for** each candidate conflict c **do**
- 7: Trace supporting conditions, values, aliases, and callee cases
- 8: Expand relevant type paths in G and re-infer in the calling context
- 9: **if** a supporting relation changes **then**
- 10: Revise its summary and invalidate dependent results
- 11: Enqueue affected caller entries in W
- 12: **end if**
- 13: Classify c using its error category and current evidence
- 14: **end for**
- 15: **end while**
- 16: Retain remaining uncovered states and undecided candidates as unresolved

4 Evaluation

We evaluate three questions: how effectively the analysis detects type errors (RQ1), how its mechanisms affect decisions and cost (RQ2), and what new errors it identifies in independent repositories (RQ3).

4.1 Experimental Setup

Benchmark. The historical benchmark contains xxx public GitHub bug-fix records from xxx Python repositories, collected over xxx. Each record identifies the buggy revision, its repaired revision, the target operation or annotation, and the evidence linking the change to the error. Table 1 summarizes the corpus. The detector analyzes each revision separately using that revision’s source and available dependencies. Patch descriptions, cross-revision comparisons, and issue metadata are reserved for assessment. The corpus emphasizes recent public type errors. We assign runtime and annotation-contract labels separately, retaining a link when a record exhibits both categories.

A target’s category and entry scope determine its eligibility. Runtime targets satisfy the protocol-failure definition in Section 2; annotation targets concern the semantic compatibility of a declaration with behavior. For every tool, results retain the full benchmark denominator and the applicable-subset denominator. Unresolved targets, timeouts, and unsupported cases have separate counts.

Baselines. We compare Mypy, Pyright [9], Pyrefly [8], Pyinder, MOPSA on its supported subset, direct LLM detection, and annotation inference followed by a type checker. The checker baselines represent declared and inferred type checking; Pyinder addresses the same detection task [13]; MOPSA provides a comparison with relational Python analysis [10]. The annotation pipeline uses xxx followed by xxx and evaluates errors in the original program separately from inconsistencies introduced by generated annotations. Direct LLM detection uses xxx with the entry and repository context specified by xxx. Tool versions and configurations are xxx. Mypy uses `--check-untyped-defs` for checking unannotated function bodies [11].

Table 1. Benchmark composition. Category counts can overlap; unique records are counted once.

Characteristic	Count or value
Historical repositories	xxx
Unique bug-fix records	xxx
Runtime type-error records	xxx
Annotation-contract error records	xxx
Records with both categories	xxx
Collection interval	xxx
Independent discovery repositories	xxx

Configuration. The analysis uses LLM xxx, decoding configuration xxx, and prompts xxx. Python and dependency versions are xxx, pyflow revision xxx, and hardware xxx. Entry selection and dependency models are xxx. The limits for explicit paths, implicit type alternatives, loop/recursive unfolding, conflict refinement, and wall-clock time are xxx. The aggregation rule for completion, resolution, and outcome counts across runs is xxx. Cost accounting includes all runs. These settings apply to the comparisons below.

Detection and precision. Paired detection requires a target report in the buggy revision and removal of the corresponding conflict in the fixed revision, with both analyses complete and the fixed target semantically resolved. For runtime errors, the buggy outcome must satisfy the entry and exception-escape criteria. For annotation-contract errors, the repaired declaration or behavior must resolve the confirmed incompatibility. A fixed target that becomes unreachable requires reachability evidence. Let N_k be the number of eligible records for category k and D_k its paired detections. We report D_k/N_k together with the full-pool rate. A record in both categories contributes once to each category and once to any unique-record aggregate.

Diagnostic precision is the fraction of reviewed reports confirmed as actual errors, including non-target errors. We report it separately for the two categories. Review follows the source, declared entry, repair or other behavioral evidence, and exception disposition; the adjudication procedure and reviewer agreement are xxx. Duplicate reports of the same root cause and category are grouped within a revision. Cross-category links are retained rather than counted as duplicate diagnoses.

Resolution and cost. Completion means the analysis process finishes within its resource limits; resolution means it establishes a target’s semantic disposition. We report both, together with unresolved, timed-out, and unsupported targets. Total cost includes call-graph construction, fact and relation extraction, composition, LLM requests, trace expansion, re-inference, and caller reanalysis. Measurements include wall-clock time, peak memory, requests, tokens, summary applications, and expanded source size. Medians and tail latencies are computed over the explicitly identified completed set; timeouts retain their consumed budget in total-cost accounting.

4.2 RQ1: Detection Effectiveness

Table 2 reports paired detection and diagnostic precision by category. Our approach detects xxx runtime targets and xxx annotation contract targets, with precision of xxx% and xxx%, respectively. The corresponding full-pool paired rates are xxx% and xxx%.

Table 3 places detection beside resolution and cost. Our analysis completes xxx% of revision analyses and resolves xxx% of eligible targets. The unresolved, timed-out, and unsupported counts

Table 2. Detection effectiveness. Paired counts and precision (%) are separated by error category. MOPSA uses its supported subset of xxx records, covering xxx% of the full pool.

Method	Runtime		Annotation contract	
	Paired	Precision	Paired	Precision
Mypy	xxx	xxx	xxx	xxx
Pyright	xxx	xxx	xxx	xxx
Pyrefly	xxx	xxx	xxx	xxx
Pyinder	xxx	xxx	xxx	xxx
MOPSA (subset)	xxx	xxx	xxx	xxx
Direct LLM	xxx	xxx	xxx	xxx
Inference + checker	xxx	xxx	xxx	xxx
Our approach	xxx	xxx	xxx	xxx

are xxx, xxx, and xxx. The median and 95th-percentile times are xxx and xxx seconds, and peak memory is xxx. Across all runs, the total number of LLM requests is xxx and token consumption is xxx.

We associate targets with the information needed to establish the checked state: input/return dependencies, condition-type relations, and attribute or global updates. These labels can overlap. For each differing outcome, we trace the operands and effects supporting the judgment. A difference is attributed to relation preservation when restoring the relation in a matched mechanism comparison changes the decision. The counts for these three dependence categories are xxx, xxx, and xxx; their respective paired detections are xxx, xxx, and xxx.

4.3 RQ2: Mechanism Effects and Cost

We compare controlled alternatives under the same entries, available repository facts, operation semantics, model configuration, and budgets. All variants use the same confirmation rules. Table 4 defines the interventions; Table 5 reports category-specific detections, unresolved targets, and total time.

State and condition preservation. Independent types, merged paths, and unknown effects isolate operand relations, branch correlations, and shared-state updates. Relative to the complete approach, their differences in paired runtime detection counts are xxx, xxx, and xxx; the corresponding differences in paired annotation-contract detection counts are xxx, xxx, and xxx. They change xxx, xxx, and xxx unresolved outcomes, respectively. Decision-level inspection links each change to the condition, value version, or object binding that reaches the target operation.

Construction and reuse. Full-body construction retains the same type candidates and source semantics, making the cost comparison sensitive to the reuse of entry-independent facts. Shared construction produces xxx use-site facts and consumes xxx seconds, compared with xxx seconds for full-body construction. Disabling implicit alternatives changes the number of represented type combinations by xxx. The corresponding differences in paired runtime and annotation-contract detection counts are xxx and xxx. The instance cache serves xxx% of eligible applications. Direct body analysis is evaluated with and without instance caching; the corresponding total costs are xxx and xxx hours.

Ordered relations and refinement. The relational-summary comparison preserves intermediate failures and error locations, so it compares how check dependencies are represented and used. It produces xxx differing target decisions and xxx differing unresolved outcomes. The selective and

Table 3. Resolution and cost. U, T, and S denote unresolved, timed-out, and unsupported targets. Time is the median per revision in seconds; total cost includes all runs. Outcome counts follow the aggregation rule in Section 4.1.

Method	Resolved (%)	U	T	S	Time	Total (h)
Mypy	xxx	xxx	xxx	xxx	xxx	xxx
Pyright	xxx	xxx	xxx	xxx	xxx	xxx
Pyrefly	xxx	xxx	xxx	xxx	xxx	xxx
Pyinder	xxx	xxx	xxx	xxx	xxx	xxx
MOPSA (subset)	xxx	xxx	xxx	xxx	xxx	xxx
Direct LLM	xxx	xxx	xxx	xxx	xxx	xxx
Inference + checker	xxx	xxx	xxx	xxx	xxx	xxx
Our approach	xxx	xxx	xxx	xxx	xxx	xxx

Table 4. Controlled mechanism comparisons. Each variant changes the named representation or procedure while retaining the common reporting criteria.

Variant	Change
Independent types	Project each case’s location relations onto independent type sets.
Merged paths	Join branch alternatives, removing condition–effect associations.
Unknown effects	Replace potentially written shared-state contents by unknown values.
Full-body construction	Construct relations from the whole function without separate entry-independent fact inference.
No implicit alternatives	Disable additional LLM type-combination proposals while retaining symbolic requirements and repository facts.
Relational summaries	Export intermediate states, normal/failure relations, and error locations; compose by linking intermediate symbols.
Direct body analysis	Analyze callee bodies at calls using the same state and operation rules.
No instance cache	Recompute instantiated results while retaining reusable summary templates.
Full callee expansion	Re-infer complete relevant callee bodies instead of selectively expanded type paths.
No dependency feedback	Assess initial candidates without revising summaries and reanalyzing dependent callers.

full-callee expansion variants use the same re-inference procedure and contextual information. They expand xxx and xxx source statements per candidate and consume xxx and xxx seconds in conflict analysis. Dependency feedback changes xxx candidates from unresolved to decided, dismisses xxx candidates after relation correction, and establishes xxx runtime and xxx annotation-contract reports. These outcomes are recorded with the responsible summary revision and the affected caller judgment.

4.4 RQ3: New-Error Discovery

The discovery corpus contains xxx repositories independent of the historical benchmark, at revisions xxx. Their selection criteria and analysis entries are xxx. Reports are reviewed using the same category and evidence rules as the historical study. Across these repositories, our approach identifies xxx runtime type errors and xxx annotation-contract errors. Their diagnostic precision is xxx% and xxx%, respectively. The numbers with independent maintainer confirmation are xxx and xxx; the numbers without that confirmation are xxx and xxx.

Table 5. Mechanism results. Runtime and contract columns report paired detections for their respective error categories. Outcome counts follow the aggregation rule in Section 4.1; total cost includes all runs.

Variant	Runtime	Contract	Unresolved	Total (h)
Complete approach	xxx	xxx	xxx	xxx
Independent types	xxx	xxx	xxx	xxx
Merged paths	xxx	xxx	xxx	xxx
Unknown effects	xxx	xxx	xxx	xxx
Full-body construction	xxx	xxx	xxx	xxx
No implicit alternatives	xxx	xxx	xxx	xxx
Relational summaries	xxx	xxx	xxx	xxx
Direct body analysis	xxx	xxx	xxx	xxx
No instance cache	xxx	xxx	xxx	xxx
Full callee expansion	xxx	xxx	xxx	xxx
No dependency feedback	xxx	xxx	xxx	xxx

For each confirmed error, the evidence record associates the source location with the entry state, relevant calls, conditional updates, and conflicting requirement or declaration. The discovery results contain xxx errors involving attribute or global updates, xxx involving condition-type relations, and xxx involving input/return dependencies. Overlapping mechanism labels remain linked to the same error record. Total analysis time over all runs is xxx hours, including xxx hours of construction and xxx hours of conflict analysis and reanalysis.

5 Threats to Validity

Construct validity. Runtime type errors and annotation-contract errors measure different properties. An open-entry failure can describe an admissible symbolic calling state without establishing a failure reachable from a repository entry. Likewise, an annotation conflict need not cause a runtime exception in an existing call. Our category-specific metrics, declared entries, and cross-category links preserve these distinctions. Manual judgments about reachability, intended interfaces, duplicate root causes, and exception handling can affect precision. The source and behavioral evidence retained with each report make these judgments inspectable; the adjudication procedure is specified in Section 4.1.

Internal validity. LLM-generated relations can omit legal type behaviors or introduce incorrect ones. Conflict-driven expansion checks disputed relations, but unresolved source semantics and incomplete context can still prevent a decision. The method does not establish soundness or completeness for arbitrary Python programs. The fixed pyflow call graph also bounds the callees available for propagation and expansion; a missing target cannot be recovered by revising a type-state relation alone. Dynamic attribute behavior, reflection, native extensions, unknown dependency effects, and bounded loop or recursive exploration can affect coverage. These cases retain unresolved or unsupported outcomes rather than being treated as confirmed absence of errors.

Ablation differences can arise from semantic coverage or budget consumption as well as the representation being studied. Matched entries, rules, and confirmation criteria help isolate these factors. We retain the causes of undecided targets and compare intermediate failures in the relational and direct-body alternatives. Differences between tools' dependency models and supported Python features remain relevant to the whole-tool comparisons.

External validity. Public GitHub fixes emphasize errors that developers discovered and chose to repair. Repository selection, issue wording, dependency availability, and the required bug-fix

pairing can therefore affect the sampled error distribution. The recent collection interval reduces exposure to older public examples but does not establish that the benchmark is absent from model training data. Model training cutoffs and corpus membership may be unknown. Independent discovery repositories test another setting, although their domains and selected entries still bound generalization.

Conclusion validity. Each experimental configuration is run three times, and effectiveness is reported using the best result. The selection metric, aggregation unit, and tie-breaking rule are xxx. This best-of-three protocol reports selected effectiveness rather than expected performance from a single run. It can favor methods with greater run-to-run variability and depends on the selection metric and aggregation unit. Consequently, the reported effectiveness cannot be interpreted as mean reliability. Retaining all run records and accounting for all three runs' cost exposes the associated variability and resource expenditure. The selected run's cost is recorded separately. Model revisions, decoding settings, and external service latency can also change outcomes and cost.

6 Related Work

Python type checking and type-error detection. Python's annotation system supplies optional type declarations, and structural protocols describe behavior shared by different classes [6, 21]. Checkers combine this interface information with inference and local narrowing [12, 18]. Empirical studies examine how gradual typing affects checker behavior and which types are needed to describe dynamic objects [19, 22]. Our analysis focuses on the state at an operation after preceding calls have changed shared locations. The relevant relation connects a calling condition to the particular update that reaches that operation.

Pyinder is a close task-level comparison. It infers types, analyzes functions under parameter-type contexts, and aggregates and filters member types to identify Python type errors [13]. We study a representation that keeps entry state, execution conditions, ordered requirements, and caller-visible updates together. Actual object bindings instantiate this representation at calls, while conflict dependencies identify the callee type paths used for re-inference. The whole-tool comparison measures detection outcomes; the controlled variants examine the roles of relation preservation and feedback.

Type inference and generated annotations. Annotation inference reconstructs interface types from code and repository context. PyTIR models entity dependencies and iteratively updates types and dependencies with checker feedback [20]. Its evaluation includes annotation accuracy and diagnostics introduced by generated annotations. Our target is the original program's operation behavior and existing annotation contracts. The state analysis uses a fixed pyflow call graph [4]; refinement changes type-state relations and their dependent judgments. The inference-plus-checker baseline evaluates how annotation generation serves this detection task, separating original errors from generated-interface inconsistencies.

Relational and compositional program analysis. Hoare logic connects assertions at program entry and exit [5]. Compositional symbolic execution reuses procedure behavior at calls [3], while abstract interpretation provides a foundation for reasoning about program states and their relations [1]. MOPSA applies abstract interpretation to Python type analysis, including state relations and control and exception behavior [10].

Our model uses these relational and compositional principles for Python type-error detection. Its cases preserve ordered checks and writes along with entry and exit assertions, so intermediate failure dependencies remain available during composition and refinement. The accompanying construction procedure shares entry-independent facts and proposes implicit type alternatives;

detection revisits disputed alternatives through selectively expanded type paths. The comparison with failure-preserving relational summaries and direct body analysis evaluates these representation and construction choices under matched rules.

7 Conclusion

We present an interprocedural type-state analysis for detecting Python type errors. The model connects function inputs and execution conditions to ordered operation requirements, shared-state updates, and exits. The analysis constructs and composes these relations, then uses conflicts to select callee type paths for expansion and contextual re-inference. Dependency-driven revisions update affected caller judgments. Runtime type errors and annotation-contract errors retain distinct confirmation criteria and results. The evaluation measures paired detection, diagnostic precision, unresolved outcomes, and complete analysis cost. The central design connects each error judgment to the state established by the calls and updates preceding the operation.

References

- [1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*. ACM, New York, NY, USA, 238–252. <https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml>
- [2] Dropbox. 2019. Our Journey to Type Checking 4 Million Lines of Python. Retrieved September 22, 2026 from <https://dropbox.tech/application/our-journey-to-type-checking-4-million-lines-of-python>
- [3] Patrice Godefroid. 2007. Compositional Dynamic Test Generation. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '07)*. Association for Computing Machinery, New York, NY, USA, 47–54. https://patricegodefroid.github.io/public_psfiles/popl2007.pdf
- [4] Zinan Gu, Haoxiang Yan, and Peisen Yao. 2026. PyFlow: An Inter-procedural Static Analysis Framework for Python. arXiv:2608.07026 doi:10.48550/arXiv.2608.07026
- [5] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580, 583. doi:10.1145/363235.363259
- [6] Ivan Levkivskiy, Jukka Lehtosalo, and Łukasz Langa. 2017. PEP 544 – Protocols: Structural Subtyping (Static Duck Typing). Retrieved September 9, 2026 from <https://peps.python.org/pep-0544/>
- [7] Meta Platforms, Inc. n.d.. Pyrefly. Retrieved September 22, 2026 from <https://github.com/facebook/pyrefly>
- [8] Meta Platforms, Inc. n.d.. Pyrefly: A Fast Python Type Checker and Language Server. Retrieved September 22, 2026 from <https://pyrefly.org/>
- [9] Microsoft. n.d.. Pyright: Static Type Checker for Python. Retrieved September 22, 2026 from <https://github.com/microsoft/pyright>
- [10] Raphaël Monat, Abdelraouf Ouadjaout, and Antoine Miné. 2020. Static Type Analysis by Abstract Interpretation of Python Programs. In *34th European Conference on Object-Oriented Programming (Leibniz International Proceedings in Informatics, Vol. 166)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:29. doi:10.4230/LIPIcs.ECOOP.2020.17
- [11] Mypy contributors. n.d.. The mypy Command Line. Mypy Documentation. Retrieved September 22, 2026 from https://mypy.readthedocs.io/en/stable/command_line.html
- [12] Mypy contributors. n.d.. Type Narrowing. Mypy Documentation. Retrieved September 16, 2026 from https://mypy.readthedocs.io/en/stable/type_narrowing.html
- [13] Wonseok Oh and Hakjoo Oh. 2024. Towards Effective Static Type-Error Detection for Python. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, 1808–1820. doi:10.1145/3691620.3695545
- [14] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, and Brent Yorgey. 2026. Hoare Logic, Part I. Software Foundations, Volume 2: Programming Language Foundations. Retrieved September 16, 2026 from <https://softwarefoundations.cis.upenn.edu/plf-current/Hoare.html> Version 7.1.
- [15] Python Software Foundation. n.d.. Data Model. Python Language Reference. Retrieved September 15, 2026 from <https://docs.python.org/3/reference/datamodel.html>
- [16] Python Software Foundation. n.d.. Execution Model. Python Language Reference. Retrieved September 16, 2026 from <https://docs.python.org/3/reference/executionmodel.html>

- [17] Python Software Foundation. n.d.. Glossary: Duck-Typing. Python Documentation. Retrieved September 16, 2026 from <https://docs.python.org/3/glossary.html#term-duck-typing>
- [18] Python typing contributors. n.d.. Type Narrowing. Python Typing Documentation. Retrieved September 16, 2026 from https://typing.python.org/en/latest/guides/type_narrowing.html
- [19] Ke Sun, Sheng Chen, Meng Wang, and Dan Hao. 2023. What Types Are Needed for Typing Dynamic Objects? A Python-Based Empirical Study. In *Programming Languages and Systems*. Springer Nature Singapore, 24–45. doi:10.1007/978-981-99-8311-7_2
- [20] Shuo Sun, Shixin Zhang, Jiwei Yan, Jun Yan, and Jian Zhang. 2025. Co-Evolution of Types and Dependencies: Towards Repository-Level Type Inference for Python Code. arXiv:2512.21591 [cs.SE] doi:10.48550/arXiv.2512.21591
- [21] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. 2014. PEP 484 – Type Hints. Retrieved September 9, 2026 from <https://peps.python.org/pep-0484/>
- [22] Wenjie Xu, Lin Chen, Chenghao Su, Yimeng Guo, Yanhui Li, Yuming Zhou, and Baowen Xu. 2023. How Well Static Type Checkers Work with Gradual Typing? A Case Study on Python. In *Proceedings of the 31st IEEE/ACM International Conference on Program Comprehension (ICPC '23)*. IEEE, 242–253. doi:10.1109/ICPC58990.2023.00039